Sistema di sensori di parcheggio IoT

Università degli Studi di Udine - Esame di Internet of Things (2020/2021)

Antonutti Davide - 141997 (email: antonutti.davide@spes.uniud.it)
Belliato Riccardo - 142652 (email: belliato.riccardo@spes.uniud.it)
Lena Emanuele - 142411 (email: lena.emanuele@spes.uniud.it)
Dumitru Alexandru Paraschiv - 141963 (email: dumitru.alexandruparaschiv@spes.uniud.it)

Sommario:

1. Introduzione **3**
1.1. Obiettivo 3
1.2. Caratteristiche del sistema 3

2. Prima soluzione architetturale **3**
2.1. Specifiche 3
2.2. Schema progettuale 4
2.4. Criticità 4

3. Soluzione finale **5**
3.1. Specifiche 5
3.2. Schema Progettuale 5
3.3. Implementazione 6
3.3.1. Microcontrollore ESP32 6
3.3.2. Microcontrollore ESP-01S 8
3.3.3. Comunicazione MQTT 8
3.3.4. Streaming Video 9
3.3.5. Interfaccia Web 10
3.3.6. Docker 11

4. Conclusioni **13**

5. Repository del progetto **14**

6. Immagini **14**

1. Introduzione

1.1. Obiettivo

L'obiettivo del progetto è quello di costruire un sistema di sensori di parcheggio che sfrutta le tecnologie IOT per fornire un servizio comparabile a quello dei sistemi in commercio.

1.2. Caratteristiche del sistema

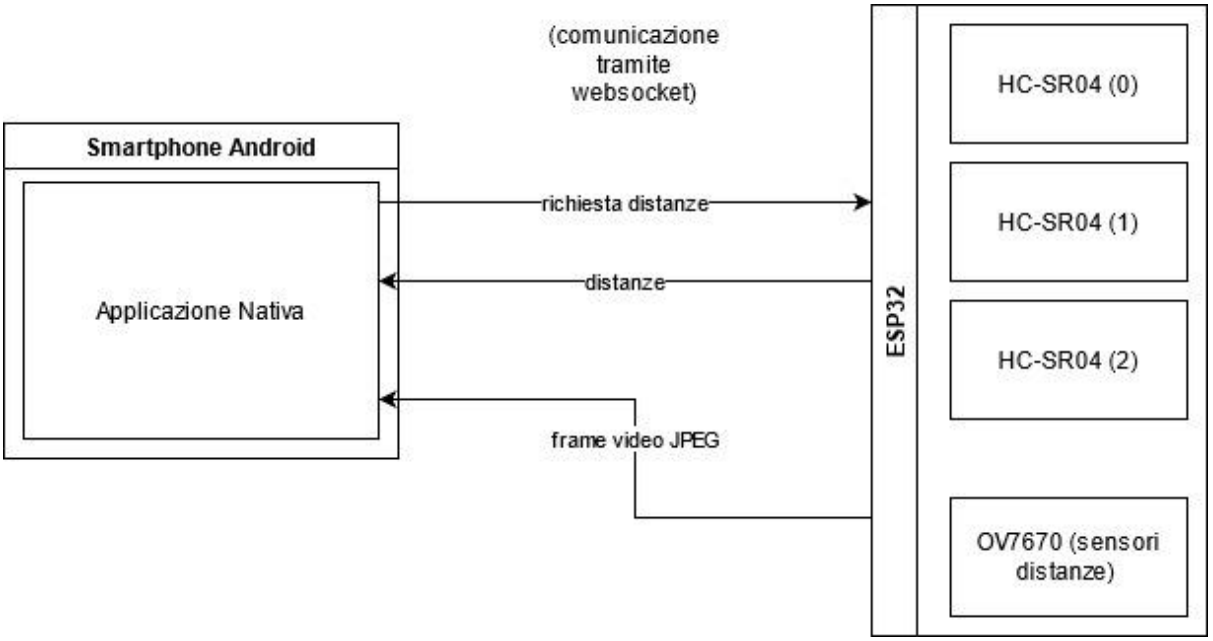
Si vuole costruire il prototipo di un sistema che assiste il guidatore nella procedura di parcheggio di un autoveicolo. Il sistema deve essere in grado di rilevare eventuali ostacoli e segnarli al guidatore in maniera opportuna. Ispirandosi ai sistemi attualmente in uso nelle moderne autovetture, si è pensato di fornire all'utente delle indicazioni su eventuali ostacoli dietro il suo veicolo attraverso il lavoro combinato di dei rilevatori di distanza e di uno streaming video.
Per quanto riguarda la sensoristica, si ipotizza di leggere le distanze attraverso sensori ad ultrasuoni (facilmente leggibili da microcontrollori) e di catturare lo streaming video con una videocamera di qualche tipo. Per l'output invece si prevede di avere un'interfaccia grafica accessibile da un qualche schermo e anche un feedback sonoro (simile a quello che già si utilizza nei sistemi in commercio).

2. Prima soluzione architetturale

2.1. Specifiche

La prima soluzione prevedeva di utilizzare un microcontrollore di tipo ESP-32 a cui erano collegati tre sensori ad ultrasuoni (HC-SR04) e una videocamera (modulo OV7670). L'idea era quella si utilizzare il processore dual-core dell'ESP-32 per gestire le due funzionalità (un core si occupava dei sensori e l'altro della videocamera). L'ESP avrebbe fatto poi da hotspot wifi per un client.
Sia i dati dei sensori a ultrasuoni che della videocamera sarebbero stati inviati al client attraverso due websocket.
Il client sarebbe stato un applicazione nativa Android, che avrebbe fatto da interfaccia al sistema. L'applicazione avrebbe presentato i dati delle distanze ed il flusso video; inoltre avrebbe prodotto un feedback sonoro (un breve segnale acustico la cui frequenza aumenta in proporzione all'abbassamento delle distanze rilevate).

2.2. Schema progettuale



2.4. Criticità

Una volta costruito un primo prototipo del sistema, si sono individuate immediatamente una serie di criticità importanti.
Le prime individuate sono di tipo prettamente tecnico, in particolare dal lato della sensoristica:

- il microcontrollore utilizzato si è rivelato non sufficientemente potente per gestire tutto il carico di lavoro. Il sistema funzionava a rilento, presentando importanti ritardi (di diversi secondi), soprattutto per il flusso video. Inoltre, periodicamente il microcontrollore si riavviava inaspettatamente al seguito del fallimento di acquisizione di un fotogramma (e ciò portava spiacevoli conseguenze, in quanto lo spegnimento del microcontrollore comportava anche la disconnessione del cellulare alla rete wifi).
- La qualità del flusso video era a dir poco scadente. Le interferenze sui collegamenti fisici tra il microcontrollore e la videocamera rendevano l'immagine sgranata e "verde".
- I sensori ad ultrasuoni, essendo posti troppo vicini tra di loro e sullo stesso lato (e non essendo di prima qualità) si disturbano a vicenda, producendo valori totalmente sballati.

Per concludere, le criticità della prima soluzione non sono state soltanto "tecniche". Proseguendo con la fruizione delle lezioni del corso di Internet of Things, ci siamo resi conto che il nostro sistema aveva anche criticità di tipo strutturale ed architetturale.
L'idea di delegare tutta la computazione ad un unico microcontrollore (che diventa quindi un "nodo centrale") e di rendere il sistema accessibile soltanto tramite un'applicazione proprietaria, viola quella che è la filosofia di distribuzione, elasticità e apertura di un sistema IoT.

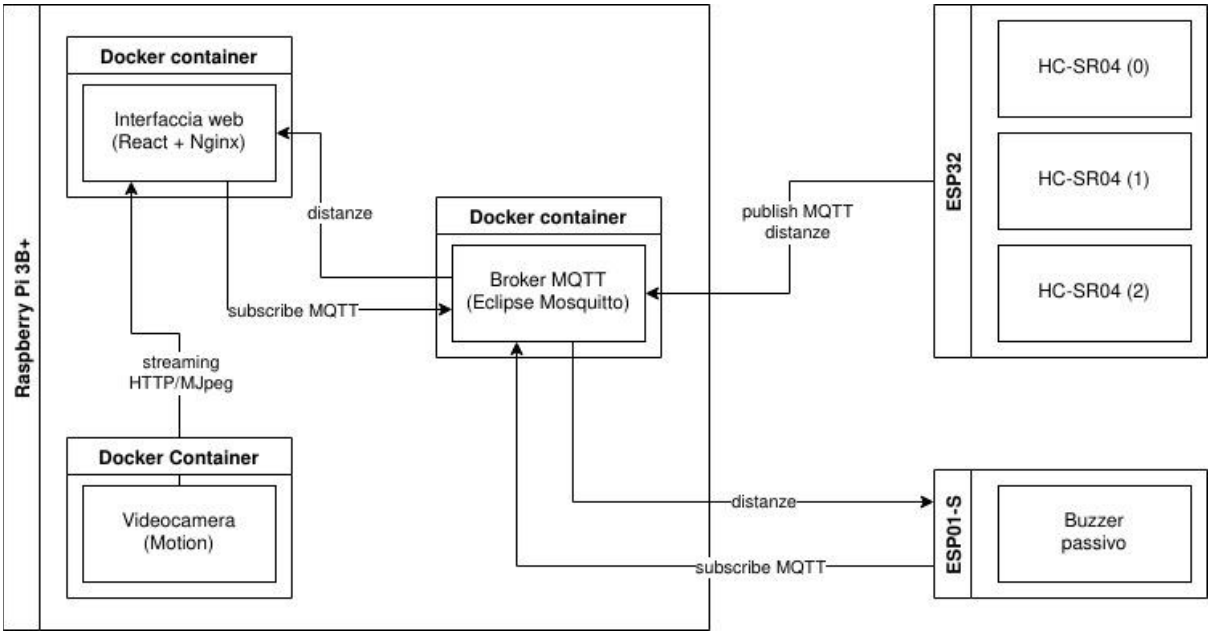
3. Soluzione finale

3.1. Specifiche

Per la nuova soluzione, si è pensato a qualcosa di più distribuito, dove una serie di componenti (hardware e software) distinti collaborano per fornire un unico servizio (in qualche modo comparabile con quello precedente). Il nuovo sistema è costituito da:

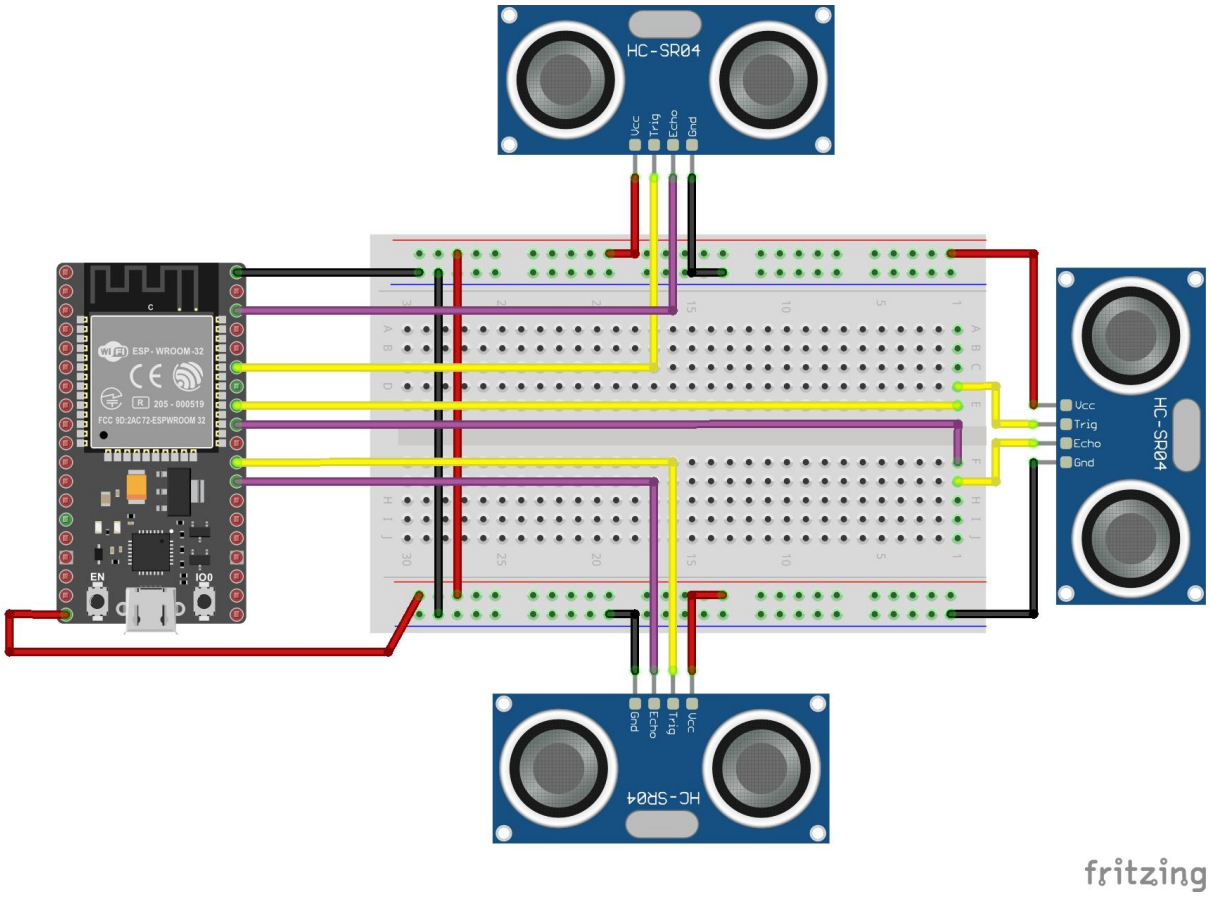
- un ESP-32, che ha solo il compito di effettuare le misure delle distanza e gestire i sensori ad ultrasuoni
- un ESP-01-S, che ascolta i cambiamenti delle distanze e pilota un buzzer per produrre il feedback sonoro (prima demandato all'applicazione)
- un Raspberry PI 3B+ che fa da hotspot wifi e fa girare una serie di componenti software
- una videocamera USB collegata al Raspberry che sostituisce la vecchia videocamera OV7670
- Il protocollo MQTT per la gestione delle comunicazioni (con il relativo broker)
- Il protocollo HTTP per lo streaming video (per i motivi che saranno spiegati successivamente).
- un'interfaccia Web responsive scritta in React, accessibile quindi da una molteplicità di dispositivi e che non richiede l'installazione su ognuno di essi
- Docker per il deployment dei vari componenti software

3.2. Schema Progettuale



3.3. Implementazione

3.3.1. Microcontrollore ESP32



Il componente HC-SR04 è un sensore per la misura delle distanze. Queste sono le sue caratteristiche tecniche (tratte dal datasheet).

Working Voltage	**DC 5 V**
Working Current	**15mA**
Working Frequency	**40Hz**
Max Range	**4m**
Min Range	**2cm**
MeasuringAngle	**15 degree**
Trigger Input Signal	**10uS TTL pulse**
Echo Output Signal	**Input TTL lever signal and the range in proportion**
Dimension	**45*20*15mm**

Il funzionamento è molto semplice: se il pin di trigger riceve un impulso di 10 µs, il sensore rilascia un treno di otto impulsi a ultrasuoni. A questo punto il sensore mantiene alto il livello di tensione del pin di echo finchè non rileva il treno di impulsi lanciato in precedenza (le onde sonore rimbalzano sugli ostacoli). A questo punto conoscendo la velocità del suono nell'aria v (circa 340 m/s) e il tempo t in cui il pin echo è rimasto nello stato HIGH, per calcolare la distanza dall'ostacolo è sufficiente applicare la formula:

$$s = \frac{t*v}{2}$$

La divisione per 2 è necessaria in quanto il tempo t ottenuto è il tempo necessario all'impulso per raggiungere l'ostacolo e tornare indietro, quindi il doppio del tempo necessario al suono per raggiungere l'ostacolo.

L'implementazione del codice per l'ESP-32 si basa sulla libreria NewPing, la quale si occupa delle operazioni descritte qui sopra e ritornare la distanza dall'ostacolo in cm. Nonostante la distanza massima misurabile dal sensore sia 4 m (una misura decisamente fuori scala per gli scopi del progetto, inoltre maggiore è la distanza massima, maggiore è il tempo utilizzato dai sensori per ottenere delle misure in assenza di ostacoli vicini), la distanza massima considerata dal progetto è 2 m, oltre la quale NewPing torna 0 cm. Se un sensore invia al microcontrollore una distanza di 0 cm, questa viene automaticamente convertita in 200 cm dallo sketch, per due motivi:

- la distanza minima misurabile dal sensore è 2 cm,
- una distanza di 0 cm in un sensore di parcheggio ha poco senso, perchè vuol dire che si ha appena fatto un incidente (e non serve certo una misurazione per accorgersene).

Le misurazioni sui singoli sensori vengono fatte in sequenza, e tra una misura e l'altra c'è una pausa di 33 ms. Questa serve per far sì che un sensore nell'effettuare una misura non venga disturbato da eventuali onde di ritorno pendenti generate da una misura precedente.

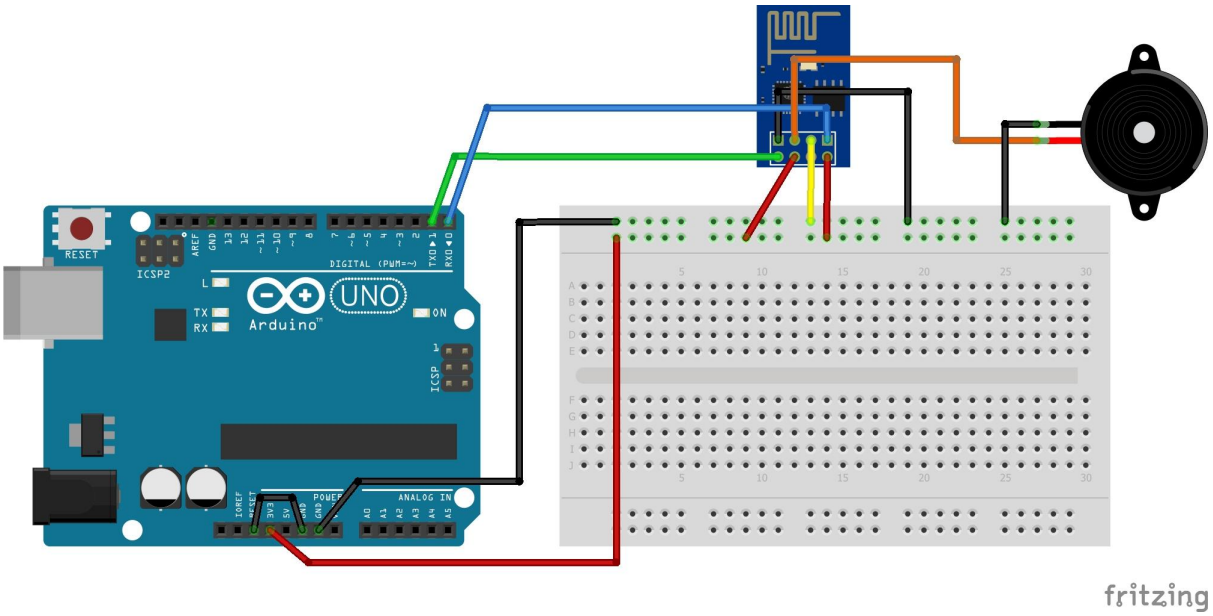
A ogni sensore è associato un topic MQTT diverso del tipo "ultrasonic/<id-sensore>", dove id-sensore è l'indice del sensore nell'array sonar dello sketch in cui sono stati dichiarati e inizializzati.

I vantaggi di questo approccio sono notevoli:

- I messaggi inviati al broker MQTT sono semplici valori numerici, non è necessario utilizzare strutture dati da serializzare su stringa (con conseguente spreco di CPU e memoria), perchè l'identificativo del sensore già presente nel topic,
- I client MQTT non devono iscriversi a tutti e tre i topic dei sensori, è sufficiente l'iscrizione al topic "ultrasonic/+" per ricevere tutte le misurazioni,
- Non dovendo aggregare i dati, non è necessario attendere che tutte e tre i sensori abbiano terminato la loro misura per inviare i dati, aumentando così la responsività dell'applicazione finale,
- Nel caso in futuro sia necessaria l'installazione di ulteriori sensori, se tutti i client sono iscritti al topic generico al secondo punto, è necessario modificare solamente lo sketch, senza intaccare le altre parti dell'architettura.

Le serie di tre misurazioni vengono effettuate a intervalli di 300 ms, per evitare di sovraccaricare eccessivamente sia la rete di sensori che i subscribers (soprattutto l'ESP01-S con il buzzer).

3.3.2. Microcontrollore ESP-01S



Il modulo ESP-01S è collegato attraverso Wi-Fi alla rete e utilizza una comunicazione MQTT per ricevere dal microcontrollore ESP32 i dati relativi alle distanze rilevate dai sensori ad ultrasuoni. Questi dati sono necessari, in quanto la scheda gestisce un buzzer passivo, un piccolo dispositivo che emette un suono periodico (beep) ad una frequenza e intermittenza scelte preventivamente. Una volta determinato il minimo tra i valori registrati dai sensori, il modulo decide se attivare la sequenza di *beeps*, e in caso il tempo di pausa tra due suoni. Assumendo che il microcontrollore riceva quasi sempre i dati delle distanze nello stesso ordine in cui sono stati inviati dall'ESP32, quando riceve un messaggio MQTT con topic "ultrasonic/0" salva il valore ricevuto come minimo. Man mano che arrivano le altre distanze, si confronta il dato in arrivo con il minimo salvato. Quando viene ricevuto l'ultimo messaggio della serie (topic "ultrasonic/2" in questo caso) si registra il valore minimo della sequenza.
Naturalmente il buzzer riduce l'intervallo di tempo tra i singoli *beeps* tanto quanto l'ostacolo si avvicina, in modo da avvisare gradualmente l'utente riguardo ad un'imminente collisione. Sono presenti quattro possibili stati per il buzzer, dallo stato di "spento" in cui non si rilevano oggetti in prossimità; passando per i due intermedi in cui viene identificato qualcosa di abbastanza vicino, ma non troppo per essere pericoloso; e infine a quello di suono continuo, dove l'oggetto è quasi a contatto col sistema.
Per utilizzare il buzzer si è fatto uso della libreria EasyBuzzer, essa permette di cambiare i parametri del buzzer per l'emissione di suoni (frequenza, durata, pausa, etc...), dopo ogni misurazione.

3.3.3. Comunicazione MQTT

Per la comunicazione, si è scelto di utilizzare un protocollo più decentralizzato ed elastico rispetto alle websocket della prima soluzione. La soluzione scelta, è stato l'utilizzo del protocollo MQTT, con broker Mosquito:

- si è installato sul Raspberry il broker MQTT Mosquito (opportunamente racchiuso in un container Docker).
- Il microcontrollore atto alla lettura dei sensori, invece che rendere disponibili le distanze su websocket, pubblica ognuna di esse su un topic MQTT.
- L'interfaccia grafica (web), si iscrive ai tre topic delle distanze e rimane in ascolto
- Il microcontrollore atto a produrre il feedback sonoro, si iscrive anch'esso ai tre topic e rimane in ascolto

La libreria utilizzata dai due sketch su ESP32 e ESP01-S per l'utilizzo di MQTT è PubSubClient. Un difetto di questa libreria è che pubblica messaggi solamente con QoS 0 (at most once), tuttavia vista l'elevata frequenza con cui questi messaggi vengono pubblicati e le dimensioni ridotte della rete di sensori, anche nelle prove effettuate questa fattispecie non ha presentato punti di criticità rilevanti.

3.3.4. Streaming Video

A differenza della soluzione precedente, lo streaming video, in questo caso, è stato dato in gestione al Raspberry. Si è scelto di far gestire il flusso al Raspberry per due motivi:

- la potenza di calcolo del Raspberry consente una gestione più stabile del flusso, oltre che una potenziale qualità maggiore
- il Raspberry, grazie alle sue porte USB, permette di utilizzare direttamente le webcam USB presenti in commercio (di qualità certamente superiore a quella "hardware" utilizzata prima; inoltre in questo modo non è necessario effettuare manualmente i collegamenti elettrici e ci si ripara quindi da potenziali disturbi).

Lo streaming video è stato gestito attraverso il software Motion, un programma open source che permette di leggere un flusso video da una videocamera (tra cui le classiche webcam USB che supportano lo standard V4L2) ed esporlo in streaming.
Motion ha la possibilità di creare uno streaming con il flusso video catturato attraverso la coppia di protocolli HTTP/M-jpeg (gli unici che supporta).
M-jpeg (abbreviazione di Motion jpeg) è un protocollo di compressione video in cui ogni frame viene compresso separatamente e inviato come immagine jpeg.

Per quanto riguarda HTTP, uno streaming M-jpeg è una risposta HTTP nel cui header viene indicato il tipo MIME **multipart/x-mixed-replace; boundary=<boundary-name>**. Questo codice indica al client che la risposta è composta da tante parti (i frame jpeg in questo caso) in arrivo in momenti diversi e ogni parte è delimitata da **<boundary-name>**.

Lo streaming video è visibile anche direttamente da browser, ed è facilmente incorporabile in una pagine web, semplicemente inserendolo all'interno di un <img src="indirizzo dello streaming" />.

Per quanto HTTP/M-jpeg siano protocolli ormai superati dai più moderni protocolli MPEG, ha dalla sua la semplicità d'uso e il fatto che sia molto leggero sia dal punto di vista della rete

che dal punto di vista dell'utilizzo delle risorse hardware. Per questi motivi è stato scelto come protocollo per lo streaming.

Per eseguire Motion sul Raspberry, abbiamo creato un'immagine Docker. Questa immagine è basata sull'immagine Docker di Ubuntu 20.04 per arm32v7 su cui viene installato Motion da apt. Siccome Motion si avvia leggendo la configurazione dal file /etc/motion/motion.conf, questa directory viene montata nel container come volume. L'immagine è stata poi compilata da una pipeline di Azure DevOps, per poter compilare con l'architettura giusta tramite l'emulatore QEMU, in assenza di un Raspberry.
Questa è la configurazione di Motion:

Opzione	**Valore**	**Descrizione**
videodevice	/dev/video0	path della webcam
stream_localhost	off	Se on lo streaming è visibile solo da http://localhost
framerate	20	numero di fps massimi della webcam
stream_maxrate	20	numero di fps massimi dello streaming
stream_port	8081	porta TCP dello stream
picture_output	off	Se on, Motion acquisisce un'immagine e la salva come file quando rileva un movimento nell'inquadratura. Se off non fa nulla.
movie_output	off	Se on, Motion salva il video acquisito come file. Se off non fa nulla.

Per poter utilizzare correttamente Motion su Docker è necessario passargli il path del webcam nel filesystem ed eseguirlo con i privilegi di amministratore.

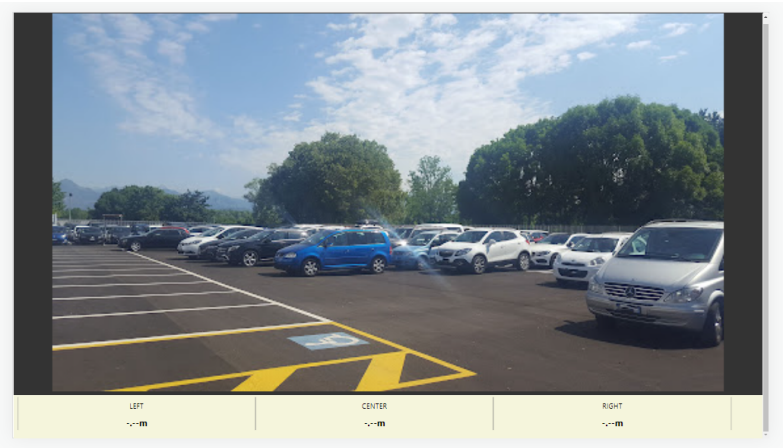
3.3.5. Interfaccia Web

L'interfaccia del nuovo sistema consiste in un'applicazione web statica, interamente "lato client" sviluppata in javascript (con React) e servita attraverso il server web statico nginx.
Dal punto di vista del codice, l'applicazione è costituita da una serie di componenti react indipendenti annidati, in particolare si hanno:

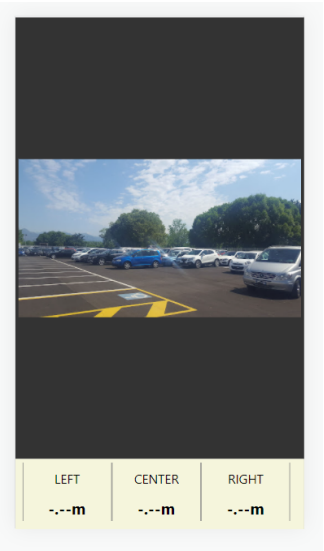
- un contenitore responsive per il video, che mostra a schermo un riquadro di proporzioni fisse che si adatta alle dimensioni dello schermo, in modo da sfruttare il più possibile lo spazio disponibile (sia per schermi verticali che orizzontali)

- tre displayer per le distanze, ciascuno iscritto tramite MQTT al suo rispettivo topic e che si aggiorna automaticamente ogni volta che viene pubblicato un nuovo valore.

Il “flusso video” è stato inserito semplicemente inserendo nel contenitore un’immagine (un tag img con l’src corrispondente all’url dello streaming).



Aspetto dell’interfaccia su schermo orizzontale



Aspetto dell’interfaccia su schermo verticale

3.3.6. Docker

Sia il broker MQTT, l’interfaccia web e il software Motion vengono serviti attraverso container Docker. Il deployment di questi avviene attraverso Docker-compose. Il suo utilizzo ha quindi permesso di creare una configurazione unica che permetta di avviare l’intero applicativo per mezzo di un unico comando.

Di seguito viene riportata il contenuto del Docker-compose.

```yaml
version: '3'

services:
    react-app:
        image: nginx:latest
        ports:
            - "80:80"
        volumes:
            - "./react-config/build:/usr/share/nginx/html"
        links:
            - mqtt

    mqtt:
        image: eclipse-mosquitto
        ports:
            - "1883:1883"
            - "8883:8883"
        volumes:
            - "./mosquitto-config:/mosquitto/config"

    motion:
        image: rik1599/motion-raspberry-container:arm32v7-latest
        ports:
             - "8081:8081"
        volumes:
             - "./motion-config:/etc/motion"
        devices:
             - "/dev/video0:/dev/video0"
        privileged: true
```

Come si può osservare sotto il campo “services”, sono presenti una serie di etichette che identificano i servizi con le relative configurazioni. Rispettivamente si ha:

- react-app: identificata l’interfaccia web
- mqtt: identifica il broker Mqtt
- motion: identifica Motion

Per quanto riguarda l’interfaccia web, viene fornita per mezzo di un immagine del web server “nginx”. Questa permette di servire una cartella statica, che è costituita dalla build dell’applicazione react. Viene mappata la porta 80 e viene effettuato un link con il container mqtt.

La configurazione del broker mqtt parte dall’immagine “eclipse-mosquitto”. Vengono mappate le porte 1883 e 8883, quest’ultima è per la websocket.

Per il software motion è stata in primis creata un’immagine per l’architettura arm a 32 bit, viene effettuata una mappatura sulla porta 8081. Inoltre il container viene eseguito in modo “privileged”.

4. Conclusioni

In questo progetto, si ha avuto modo di mettere a confronto due soluzioni per lo stesso problema: una "monolitica e chiusa" e una distribuita. Il risultato più soddisfacente è stato indubbiamente portato dalla seconda soluzione, in quanto ci ha permesso di risolvere tutti i problemi evidenziati con la prima e di ottenere un sistema funzionante al 100%.
Le scelte architetturali della prima soluzione, a posteriori, sono da considerarsi non adatte ad un sistema di tipo IoT, in quanto:

- si sono sopravvalutate le capacità di computazione dell'hardware
- si ha delegato il compito di "nodo centrale" di una rete ad un dispositivo troppo semplice e instabile, che dovrebbe invece essere utilizzato solo per adempiere a piccoli compiti molto specifici (la lettura di un sensore, il controllo di un dispositivo di output, ecc…)
- si ha fatto una scelta di tecnologie e protocolli poco consona all'ambiente IoT (le websocket per la comunicazione, un'applicazione Android nativa per l'interfaccia)

La seconda soluzione invece ci ha permesso di applicare a pieno le potenzialità delle tecnologie IoT:

- I microcontrollori in questo caso sono stati utilizzati correttamente, senza attribuire loro responsabilità troppo grosse
- Il centro della rete è un dispositivo più solido, meno soggetto al rischio di crash improvvisi e più facilmente configurabile per aggiungere funzionalità.
- La scelta del protocollo MQTT rende il sistema più aperto all'aggiunta di nuovi componenti (a questo punto, ad esempio, sarebbe molto facile aggiungere un display pilotato da un terzo microcontrollore).
- L'interfaccia web rende possibile l'accesso da parte di più dispositivi, anche eterogenei
- L'utilizzo di Docker per il deployment rende più semplice distribuire il software (ad esempio, in caso di "produzione in serie" del sistema)

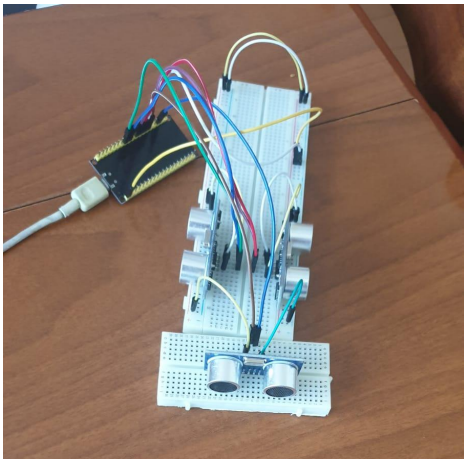
La seconda soluzione risulta vincente anche dal punto di vista del "costo di sviluppo", in quanto, vista l'estrema semplicità dei singoli componenti, è stata più facile da sviluppare. Il fatto di avere più componenti molto semplici (che comunicano con protocolli molto standardizzati) ci ha permesso di dividerci più facilmente i lavori ed arrivare piuttosto velocemente ad un prototipo funzionante.

Forse l'unico aspetto in cui la prima soluzione è migliore è il costo dei componenti. La prima soluzione difatti risulta più economica (partendo sempre dal presupposto che l'utilizzatore abbia già uno smartphone).

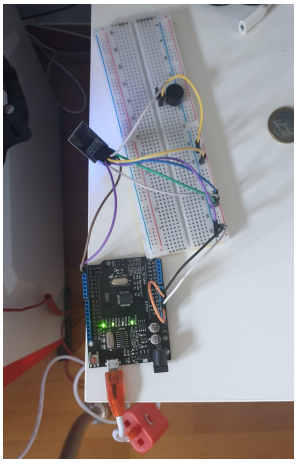
5. Repository del progetto

https://github.com/rik1599/IoT2021_UltrasonicESP
https://github.com/rik1599/motion-raspberry-docker
https://github.com/emanuelena49/iot-parking-sensors-ui
https://github.com/alexandruparaschivdumitru/iot-parking-sensor-config
https://github.com/Antonutti99/iot-parking-app-buzzer

6. Immagini



Circuito dell'ESP-32 con i sensori ad ultrasuoni



Circuito dell'ESP-01 con il buzzer

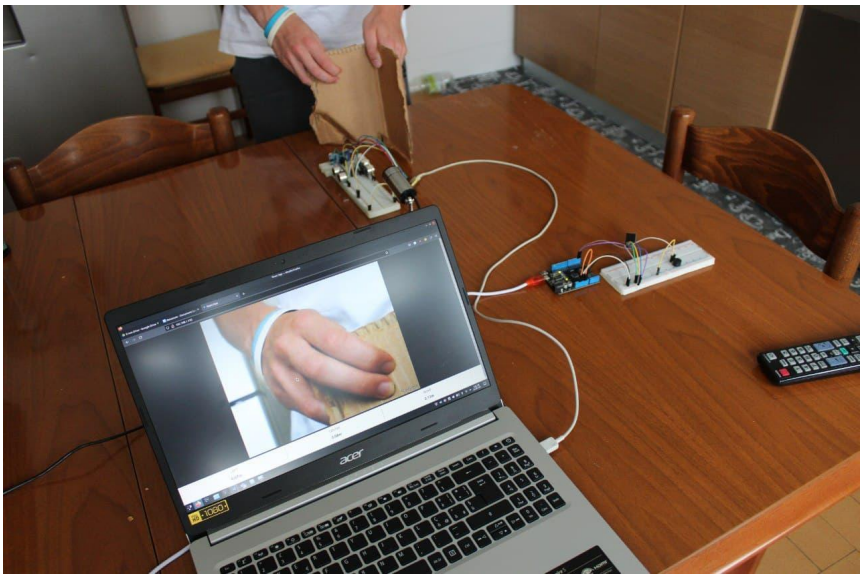


Foto del sistema in funzione. Si possono osservare i due circuiti, la webcam collegata al Raspberry (nascosto dietro il monitor) e l'interfaccia web visibile dal browser del computer.